

ResNet-18 comparative analysis of various activation functions for image classification

Gaurav Kumar Pandey

Computer Science and Engineering Department
Birla Institute of Technology Mesra,
Ranchi, India
gaurav.pandey1812@gmail.com

Sumit Srivastava

Computer Science and Engineering Department
Birla Institute of Technology Mesra,
Ranchi, India
sumit@bitmesra.ac.in

Abstract—Deep neural network and Machine learning are a latest emerging concept in the field of data science. Due to multi-layer hierarchical feature extraction in conjunction with control variables like number of hidden layers, activation functions, and variable parameters like learning rates, initial weights, and decay functions, deep network models perform better than machine learning techniques. While most of these parameter control the learning dynamics or complexity of representation a neural network can deal with, it is only activation function which introduces non-linearity in a network and current state of activation function poses multiple challenges to both practitioners and researchers some of which are:

- Vanishing & Exploding gradients during back-propagation
- Zero-mean and range of outputs
- Compute complexity of function
- Predictive performance

Due to this reason our objective in current work is focused to explain with reasoning and experiments the landscape of activation functions available. According to a recent study, we have enough cutting-edge activation functions to modify the architecture of the well-known deep network model. Building on top of widely adopted ResNet-18 network architecture in this study. Subsequently, we evaluate the effectiveness of ResNet-18 for image classification using various activation functions.

Index Terms—Activation Function, Neural Networks, Machine learning, ReLU, CIFAR-10, Accuracy

I. INTRODUCTION

For deep neural networks to function properly, activation functions [1] are essential. They give the model non-linearity, enabling it to approximate intricate, non-linear interactions between inputs and outputs. A deep neural network would essentially be a linear regression model without activation functions. Tremendous improvements [2] have been witnessed in the representation and recognition performance of the neural networks with deep architectures, due to which the landscape of computer vision and natural language processing have been noticeably changed.

CNNs are trained using a large dataset of labeled images, which allows them to learn to recognize different objects and features within images. This makes them useful for a variety

of tasks, such as image classification, object detection, and image segmentation. During training, the network is shown a large dataset of labeled images and learns to recognize patterns and features within the images that are associated with different categories. Once trained, the CNN can classify new images by analyzing their features and comparing them to the learned patterns from the training data.

The objective of this study is to evaluate the performance of the ResNet-18 architecture with various activation functions. The ResNet-18 architecture is a popular deep learning model commonly used for image classification tasks. In this study, we will investigate the effects of using different activation functions such as ReLU, ELU, Gelu, Swish, Softplus, Softsign and Tanh on the accuracy and convergence speed of the ResNet-18 model.

The aim of this study is to identify the best activation function for the ResNet-18 architecture that can achieve the highest accuracy on a given image classification task while maintaining fast convergence speed. The experimental results will be compared and analyzed to provide insights into the effects of activation functions on the performance of deep learning models.

To accomplish this objective, we will conduct experiments on a standard image classification dataset and measure the accuracy and training time of the ResNet-18 model using different activation functions. The results will be analyzed to determine the best activation function for the ResNet-18 architecture. The revolutionary changes have resulted from the next crucial elements including building more powerful models accumulating large-scale data sets developing higher-performance hardware designing more effective regularisation techniques and so on.

II. ACTIVATION FUNCTION

The role of activation function [3] in the field of neural networks is very crucial. There are the following properties that should be held by activation functions. 1. non-linearity [4] is important to improve the training convergence of the

Identify applicable funding agency here. If none, delete this.

network, 2. It should not increase the computational complexity of the model extensively, 3. Gradient flow should not be disturbed during the training, 4. The distribution of data should be in such a manner to facilitate better training of the network. Below are the following acting activation functions which we are going to use in this research paper.

- Tanh
- Softsign
- ReLU
- SoftPlus
- ELU
- GELU
- Swish

A. Tanh

The Tanh (hyperbolic tangent) [5] activation function is a commonly used non-linear activation function in neural networks shown in **figure1**. Here are some of its merits:

- Range
- Non-linear
- Smoothness
- Zero-centered

The inputs are also compressed using the Tanh function but in the range [-1, 1]. The Tanh function shares with the Logistic Sigmoid function certain disadvantages, including diminishing gradient and computing cost.

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (1)$$

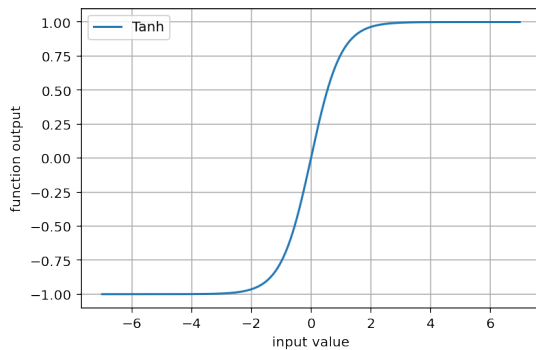


Fig. 1. Depiction of Tanh function

B. Softsign

The Softsign [6] activation function is a smooth and continuous function that is commonly used in neural networks shown in **figure2**. It is defined as:

$$f(x) = x / (1 + \text{abs}(x)) \quad (2)$$

where $\text{abs}(x)$ is the absolute value of x .

The Softsign activation function has the following properties:

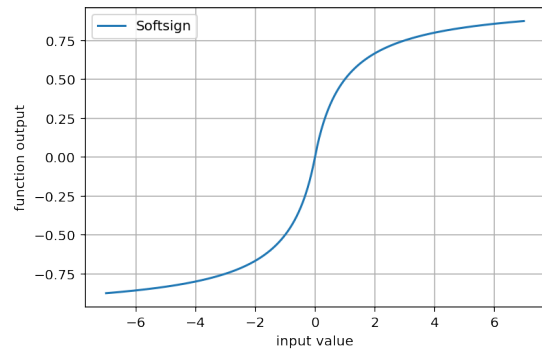


Fig. 2. Depiction of SoftSign function

- It maps any real value to a value between -1 and 1, so it is useful for normalizing the input to the next layer.
- It is a smooth function, which means that it has a continuous first derivative. This makes it easier to optimize using gradient-based methods.
- It is zero-centered, which means that the output has a mean of zero.

Compared to other activation functions, such as the sigmoid or tanh functions, the Softsign function is less prone to the vanishing gradient problem. However, it can still suffer from the exploding gradient problem, especially for large inputs. The Softsign activation function is not used as frequently as other activation functions, but it can be useful in some cases where normalization or smoothness is important.

C. ReLU(Rectified linear unit)

In deep neural networks, the rectified linear unit (ReLU) [7] is a popular activation function. Its job is to make the network non-linear so that it can learn deep, non-linear correlations between the inputs and outputs. Rectified Linear Unit (ReLU) [7] is a straightforward function using zeros for negative input and the identity function for positive input as follows:

$$f(x) = \begin{cases} 0, & \text{if } x < 0 \\ x, & \text{if } x \geq 0 \end{cases} \quad (3)$$

ReLU is a simple and efficient activation function which introduced non-linearity in a layer's response. Which allows for modeling non-linear decision boundaries with the help of piecewise linear units, where each ReLU contributes to a piece of this boundary.

Also, the unit slope of ReLU for positive part mitigates the vanishing gradient problem commonly known to exist in activations functions prior to ReLU like TanH. As shown in **figure3** When the input to the ReLU function is positive, the function returns the input value as it is. When the input is negative, the function returns 0. This means that the ReLU function is essentially a threshold function, and any input below the threshold is discarded. One potential issue with ReLU is that it can result in the "dying ReLU" problem, where a neuron with a negative output during training becomes

stuck and never activates again. To mitigate this, variations of ReLU such as Leaky ReLU and ELU have been developed.

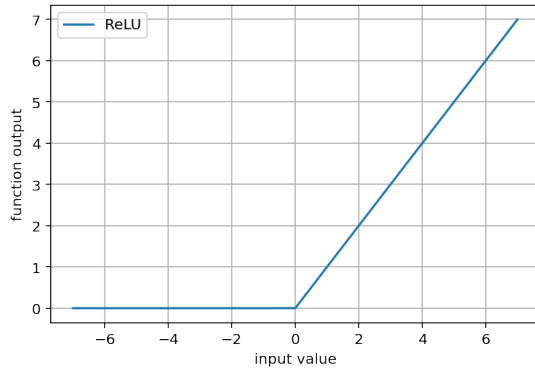


Fig. 3. Depiction of Relu function

While there exist variants of ReLU known as Leaky-ReLU [8] which has a linear nature with non-unit slope for negative values. A super-set of Leaky-ReLU can be evaluated using simple ReLU over two neural units and hence not included in our study.

D. SoftPlus

The Softplus activation function [9] is a smooth approximation of the ReLU function, and is defined as:

$$f(x) = \log_e(1 + e^x) \quad (4)$$

This code uses the $\log_e$ function to represent the natural logarithm, and the e^x notation to represent the exponential function. You can modify this code to use different variable names or adjust the formatting to fit your needs.

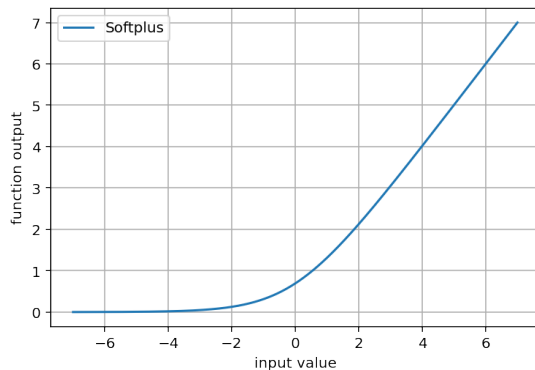


Fig. 4. Depiction of Softplus function

E. ELU

The Exponential Linear Unit (ELU) [10] was proposed as an improvement over the commonly used rectified linear unit (ReLU) activation function. The Rectified Linear Unit (ReLU) activation function, which has been widely employed in deep learning due to its simplicity and efficiency, was replaced by the Extended Linear Unit (ELU). While "dying ReLU," where

the neuron becomes stuck in a state of zero activation and ceases learning, is a known issue with ReLU, ELU attempts to address it by permitting negative values in the activation function. In the ELU activation function, the output is non-zero for negative values of x , which helps to prevent dead neurons and can lead to faster convergence and better overall performance.

$$ELU(x) = x \text{ if } x > 0 \text{ else } \alpha * (e^x - 1) \quad (5)$$

where x is the input to the activation function, and α is a hyperparameter that controls the output for negative values of x .

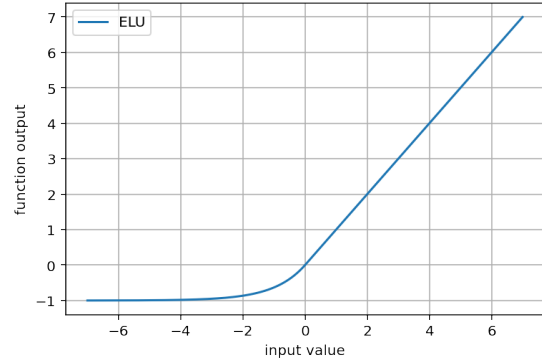


Fig. 5. Depiction of ELU function

ELU activation function has response values for negative values which enhances zero-mean property unlike plain ReLU to improve learning and predictive performance. But also it provides for deactivation for highly negative values which is not possible for Leaky-ReLU and related non-saturating activations on negative end.

However, the main disadvantage of the ELU activation function is that it is computationally more expensive than the ReLU since it involves the exponential function.

F. GELU

GELU (Gaussian Error Linear Units) [11] is an activation function that is commonly used in deep neural networks. It was introduced in the paper "Gaussian Error Linear Units (GELUs)" by Hendrycks and Gimpel in 2017. The GELU activation function is defined as:

$$GELU(x) = 0.5x \left(1 + \tanh \left[\sqrt{2/\pi} (x + 0.04471x^3) \right] \right) \quad (6)$$

The GELU activation function has several desirable properties. It is a smooth function, meaning it has a continuous first derivative. It is also non-monotonic, which means that it can model complex functions. The GELU function has been shown to be effective in deep neural networks, particularly in natural language processing (NLP) tasks, and has been found to outperform other activation functions such as the sigmoid and tanh functions. Finally, the GELU activation function has been shown to perform well in a variety of deep-learning tasks. One disadvantage of the GELU activation function is that it

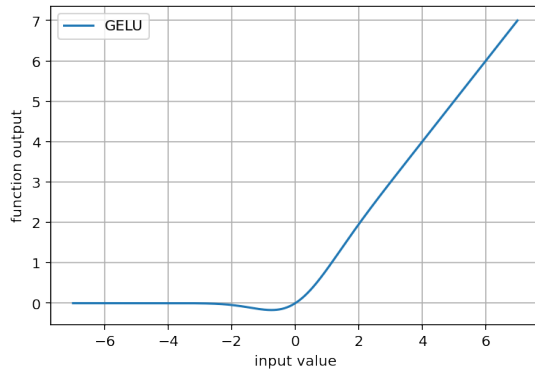


Fig. 6. Depiction of GELU function

is computationally more expensive than some other activation functions, such as the ReLU (Rectified Linear Unit) activation function. However, its improved performance in some tasks may make it worth the additional computational cost.

G. Swish

The Swish activation function [12] is a relatively new activation function proposed by Google researchers in 2017. It is defined mathematically as follows:

$$\text{Swish}(x) = x * \text{sigmoid}(\beta * x) \quad (7)$$

where β is a scalar parameter that controls the shape of the function.

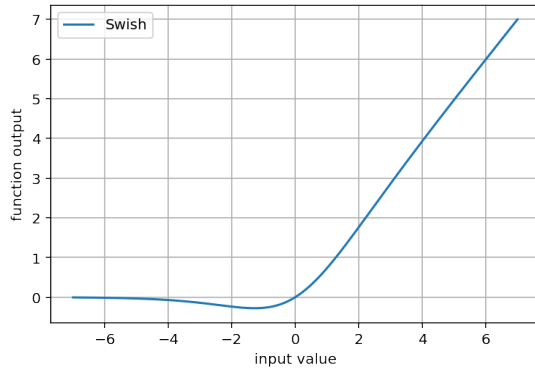


Fig. 7. Depiction of Swish function

The Swish function has several properties that make it an interesting alternative to commonly used activation functions such as ReLU (Rectified Linear Unit) or sigmoid:

- Smoothness: The Swish function is smooth, which means it has continuous derivatives, making it easier to optimize using gradient-based methods.
- Non-monotonicity: The Swish function is non-monotonic, which means it is not strictly increasing or decreasing over its entire domain. This can help to capture more complex patterns in data.
- Computational efficiency: The Swish function can be computed efficiently using standard operations such as

multiplication and sigmoid, making it easy to implement in neural networks.

Recent studies have demonstrated that on a range of tasks, including image classification, audio recognition, and natural language processing, the Swish activation function can outperform ReLU and other activation functions. To determine the most effective activation function for a given job, it is vital to experiment with various activation functions as its effectiveness might vary based on the dataset and the particular network architecture being employed.

III. CIFAR-10 DATASET

A well-known image classification dataset called CIFAR-10 [13] has 60,000 32x32 color pictures divided into 10 groups with 6,000 images each. The ten categories are as follows: truck, frog, horse, ship, bird, cat, deer, vehicle, airplane, and automobile.

10,000 test images and 50,000 training images make up the dataset. The training set is then split into 5 batches of 10,000 images each. The test set is made up of 10,000 images in one batch.

CIFAR-10 is a widely used benchmark dataset for image classification tasks, and has been used to evaluate the performance of many machine learning algorithms, including deep neural networks. It is considered a challenging dataset due to the small image size, low resolution, and high variability in appearance of objects within each class.

IV. RESNET-18

ResNet-18 [14] has a total of four residual blocks, each consisting of two convolutional layers and one identity shortcut. The first convolutional layer in each block has 64 filters and a kernel size of 3x3, while the second convolutional layer also has 64 filters and a kernel size of 3x3.

The identity shortcut connects the input of each residual block directly to its output and is used to add the input to the output of the second convolutional layer in each block. The idea behind this is that the identity shortcut provides a shortcut for gradients to flow directly from the output of the block to the input, which helps to mitigate the problem of vanishing gradients in deep neural networks.

As shown in **Figure 8** ResNet-18 [15] also includes a max pooling layer with a 3x3 kernel and stride 2, which is used to downsample the input before it is passed through the residual blocks. The final output is created by passing the output of the last residual block via a softmax activation function, a fully connected layer, and a global average pooling layer.

A. Resnet-18 Methodology

ResNet-18 is a convolutional neural network architecture that Microsoft Research Asia unveiled in 2015. It is famous for its capability to efficiently train very deep neural networks and is developed for image classification jobs. ResNet-18's technique may be summarised as follows:

- Input Layer: The network takes an input image of size 224x224 pixels.

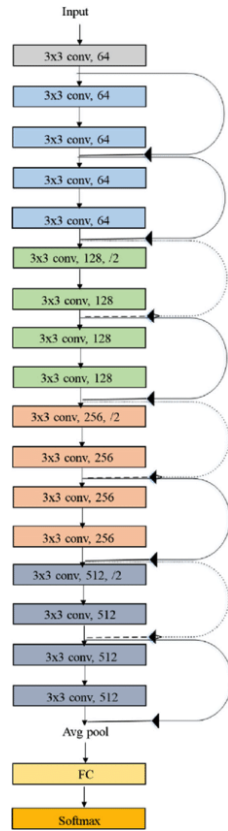


Fig. 8. Original ResNet-18 Architecture

- **Convolutional Layers:** The first layer is a 7x7 convolutional layer with a stride of 2, followed by a max pooling layer with a stride of 2. This is followed by four sets of convolutional layers, each with two 3x3 convolutional layers. Each set of convolutional layers is followed by a shortcut connection.
- **Shortcut Connections:** The shortcut connections skip one or more convolutional layers and connect directly to a later layer in the network. This helps to prevent the vanishing gradient problem and allows the network to train deeper.
- **Global Average Pooling:** The output of the final convolutional layer is passed through a global average pooling layer, which averages the feature maps over the spatial dimensions.
- **Fully Connected Layer:** The global average pooling layer is followed by a fully connected layer with 1000 units, which produces the final output of the network.
- **Softmax Activation:** The output of the fully connected layer is passed through a softmax activation function, which produces a probability distribution over the 1000 classes in the ImageNet dataset.

The ResNet-18 architecture has a total of 18 layers, which makes it a relatively shallow network compared to some other deep learning architectures. However, the use of shortcut connections allows it to train much deeper than other networks

of similar size. This makes it a powerful tool for image classification tasks.

B. Resnet-18 on CIFAR 10 dataset

ResNet-18 is a deep neural network architecture that may be used with different datasets, such as CIFAR-10, but was initially developed for image classification tasks on the ImageNet dataset. A dataset called CIFAR-10 has 60,000 32x32 colour images divided into 10 classes with 6,000 images each. We train ResNet-18 on CIFAR-10 using TensorFlow, we can follow these steps:

- Load the CIFAR-10 dataset and preprocess the images. We can use the `tf.keras.datasets` package
- Define the ResNet-18 architecture. We can create the model using the `tf.keras.Sequential` API and add the ResNet-18 layers using the `tf.keras.layers` package
- Define the loss function and optimizer:
`loss_fn = tf.keras.losses.CategoricalCrossentropy()` [16]
`optimizer = tf.keras.optimizers.SGD` [17](lr=0.1, momentum=0.9, decay=5e-4)
- Compile the model
- Train the model

During training, the model will print the loss and accuracy on the training and validation sets for each epoch. After training is complete, the final accuracy on the test set will be printed.

C. Performance of resnet-18 over different AFs

it is possible to use different activation functions in ResNet-18 to achieve different performance characteristics. It's important to note that using different activation functions in ResNet-18 can affect the network's performance and training time. It may require adjusting the learning rate, optimizer, and other hyperparameters to achieve optimal results.

V. EXPERIMENTAL RESULTS

ResNet-18 is a convolutional neural network architecture originally developed for the ImageNet dataset. However, it can also be used for other image classification tasks, such as the CIFAR-10 dataset. In our experiment, we are using Google Colab [18] to run python code, and we are using different libraries like Numpy, Pandas, Matplotlib [19], and Keras [20]. Colab is a great platform for displaying experimental results, as it allows us to easily share our code and visualizations with others. In the below table we can see that performance of resnet-18 can be improved if we use the ELU activation function. Here are some experimental results for ResNet-18 over different activation functions on the CIFAR-10 dataset in **Table I** shows the performance of resnet 18 over different activation function.

VI. CONCLUSION

The activation function is a key component of the ResNet-18 architecture, as it helps to introduce non-linearity into the network and enables it to learn complex patterns in the data. The choice of activation function can have a significant impact on the performance of the ResNet-18 model. In general, ReLU

S. No.	Model	Activation Function	Dataset	Accuracy
1	Resnet 18	Tanh	CIFAR 10	68.72
2	Resnet 18	Softsign	CIFAR 10	66.89
3	Resnet 18	ReLU	CIFAR 10	69.55
4	Resnet 18	SoftPlus	CIFAR 10	65.79
5	Resnet 18	ELU	CIFAR 10	72.86
6	Resnet 18	GELU	CIFAR 10	71.18
7	Resnet 18	Swish	CIFAR 10	70.63

TABLE I
PERFORMANCE OF RESNET-18 WITH ACTIVATION FUNCTIONS

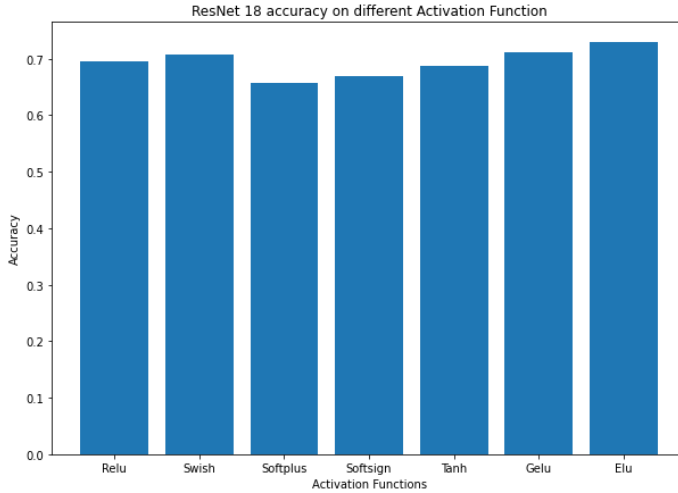


Fig. 9. A bar chart between Loss and Activation Function

is the most commonly used activation function in ResNet-18 and other deep learning architectures, as it has been shown to perform well in practice and is computationally efficient. However, other activation functions may be more suitable for certain tasks or datasets.

ResNet-18's activation function selection can significantly affect the model's performance. Although ReLU is frequently used and frequently performs well, some applications or datasets may benefit more from the usage of other activation functions. In order to choose the appropriate activation function for a particular activity, it is crucial to experiment with several activation functions and assess their effectiveness. Based on the data presented in this paper **Figure9** and **Figure10**, we may draw the conclusion that the activation function and other factors affect the resnet-18's ability to identify objects. We tested the resnet-18 network using seven different activation functions, and the greatest results came from the ELU, GELU, and Swish activation functions. We may get the conclusion that Resnet-18 performs the best with ELU, scoring 72.86% on the CIFAR-10 dataset.

REFERENCES

- [1] Andrinandrasana David Rasamoelina, Fouzia Adjailia, and Peter Sinčák. A review of activation function for artificial neural network. In *2020 IEEE 18th World Symposium on Applied Machine Intelligence and Informatics (SAMI)*, pages 281–286, 2020.
- [2] Shiv Ram Dubey, Satish Kumar Singh, and Bidyut Baran Chaudhuri. Activation functions in deep learning: A comprehensive survey and benchmark. *Neurocomputing*, 2022.
- [3] Bin Ding, Huimin Qian, and Jun Zhou. Activation functions and their characteristics in deep neural networks. In *2018 Chinese Control And Decision Conference (CCDC)*, pages 1836–1841, 2018.
- [4] Dabal Pedamonti. Comparison of non-linear activation functions for deep neural networks on mnist classification task. *arXiv preprint arXiv:1804.02763*, 2018.
- [5] S. Marra, M.A. Iachino, and F.C. Morabito. Tanh-like activation function implementation for high-performance digital neural systems. In *2006 Ph.D. Research in Microelectronics and Electronics*, pages 237–240, 2006.
- [6] Abien Fred Agarap. Deep learning using rectified linear units (relu). *arXiv preprint arXiv:1803.08375*, 2018.
- [7] Bing Xu, Naiyan Wang, Tianqi Chen, and Mu Li. Empirical evaluation of rectified activations in convolutional network. *arXiv preprint arXiv:1505.00853*, 2015.
- [8] Hao Zheng, Zhanlei Yang, Wenju Liu, Jizhong Liang, and Yanpeng Li. Improving deep neural networks using softplus units. In *2015 International Joint Conference on Neural Networks (IJCNN)*, pages 1–4, 2015.
- [9] Dork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. Fast and

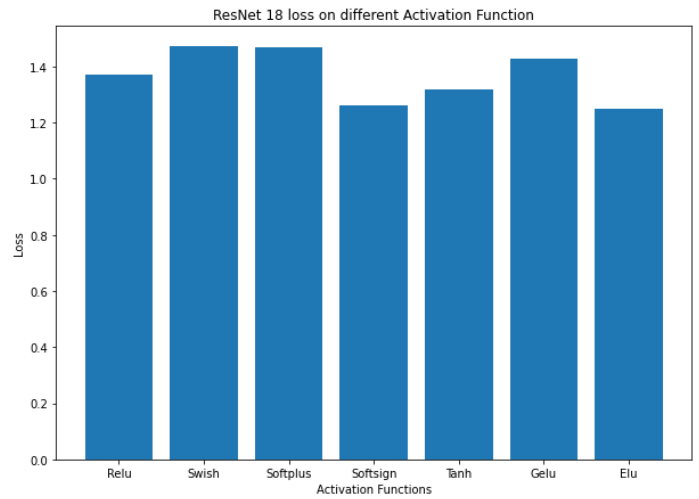


Fig. 10. A bar chart between Accuracy and Activation Function

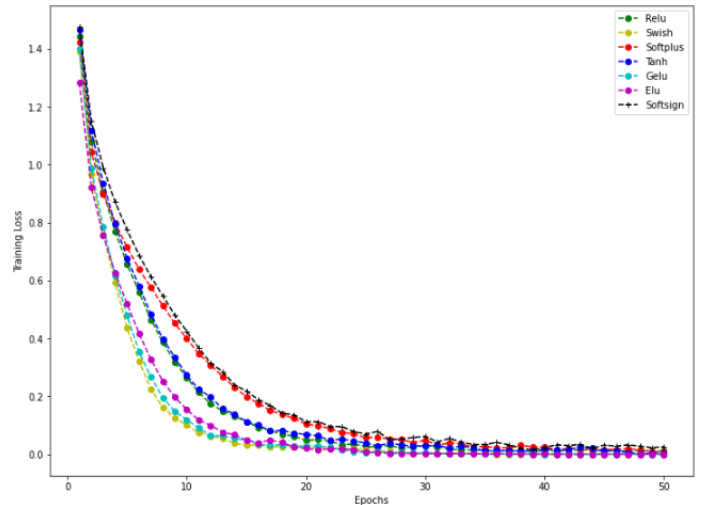


Fig. 11. A Graph between Training Loss and Epoch

- accurate deep network learning by exponential linear units (elus). *arXiv preprint arXiv:1511.07289*, 2015.
- [11] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.
 - [12] Prajit Ramachandran, Barret Zoph, and Quoc V Le. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017.
 - [13] Will Cukierski. Cifar-10 - object recognition in images, 2013.
 - [14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
 - [15] Farheen Ramzan, Muhammad Usman Khan, Asim Rehmat, Sajid Iqbal, Tanzila Saba, Amjad Rehman, and Zahid Mehmood. A deep learning approach for automated diagnosis and multi-class classification of alzheimer's disease stages using resting-state fmri and residual neural networks. *Journal of Medical Systems*, 44, 12 2019.
 - [16] Zhilu Zhang and Mert Sabuncu. Generalized cross entropy loss for training deep neural networks with noisy labels. *Advances in neural information processing systems*, 31, 2018.
 - [17] Sebastian Ruder. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*, 2016.
 - [18] Ekaba Bisong. *Google Colaboratory*, pages 59–64. Apress, Berkeley, CA, 2019.
 - [19] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
 - [20] Francois Chollet et al. Keras, 2015.